Programación Orientada a Objetos en Java

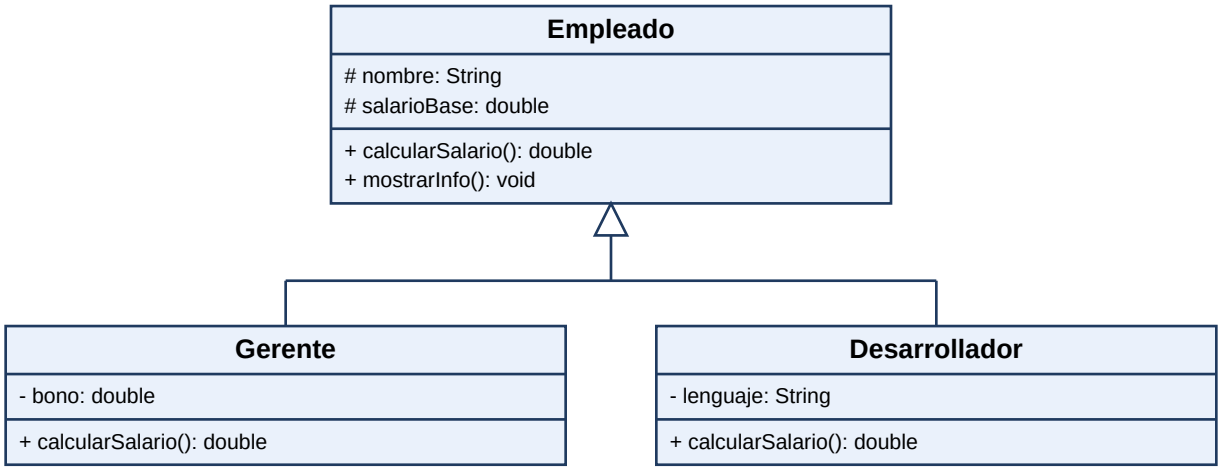
Herencia, clases abstractas, interfaces, dependencia y asociaciones

1. Repositorio en GitHub

https://github.com/TU_USUARIO/TU_REPOSITORIO

2. Ejemplo de herencia

La herencia permite que una clase hija (**Gerente**, **Desarrollador**) reutilice los atributos y métodos de una clase padre (**Empleado**) con `extends`, y que redefina comportamiento con `@Override`. Relación *"es un"*.




```
public class Empleado {
    protected String nombre;
    protected double salarioBase;

    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    public double calcularSalario() {
        return salarioBase;
    }

    public void mostrarInfo() {
        System.out.println(nombre + " gana $" + calcularSalario());
    }
}
public class Gerente extends Empleado {
    private double bono;

    public Gerente(String nombre, double salarioBase, double bono) {
        super(nombre, salarioBase);
        this.bono = bono;
    }

    @Override
    public double calcularSalario() {
        return salarioBase + bono;
    }
}
```

```
public class Desarrollador extends Empleado {
    private String lenguaje;

    public Desarrollador(String nombre, double salarioBase, String lenguaje) {
        super(nombre, salarioBase);
        this.lenguaje = lenguaje;
    }

    @Override
    public double calcularSalario() {
        return salarioBase * 1.10;   // 10 % adicional
    }
}
// Uso (polimorfismo)
Empleado e1 = new Gerente("Laura", 5000000, 1000000);
Empleado e2 = new Desarrollador("Carlos", 4000000, "Java");
e1.mostrarInfo();   // Laura gana $6000000.0
e2.mostrarInfo();   // Carlos gana $4400000.0
```

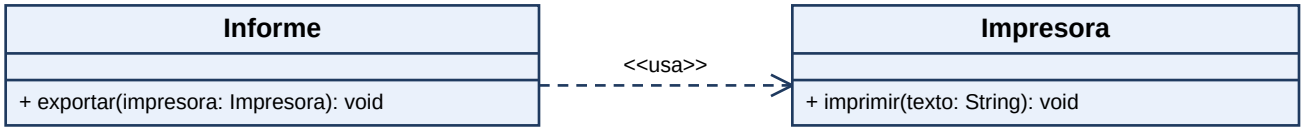
3. Diferencia entre clase abstracta e interfaz

Aspecto	Clase abstracta	Interfaz
Palabra clave	`abstract class`	`interface`
Herencia	Una sola (`extends`)	Varias (`implements`)
Métodos	Abstractos y concretos	Abstractos; también `default` y `static` (Java 8+)
Atributos	Cualquier tipo, con estado	Solo constantes (`public static final`)
Constructor	Sí	No
Modificadores	Cualquiera (`private`, `protected`...)	Los métodos son públicos
Instanciable	No	No
Propósito	Base común con código compartido ("es un")	Contrato de capacidades ("puede hacer")

```
public abstract class Figura {            // clase abstracta
    protected String color;
    public Figura(String color) { this.color = color; }
    public abstract double area();        // sin cuerpo: la hija lo implementa
    public void mostrarColor() { System.out.println(color); }
}
public interface Dibujable {              // interfaz
    void dibujar();                       // public abstract implicito
    default void info() { System.out.println("Soy dibujable"); }
}
// Extiende UNA clase abstracta e implementa VARIAS interfaces
public class Circulo extends Figura implements Dibujable {
    private double radio;
    public Circulo(String color, double radio) { super(color); this.radio = radio; }
    @Override public double area() { return Math.PI * radio * radio; }
    @Override public void dibujar() { System.out.println("Dibujando circulo " + color); }
}
```

4. Ejemplo de dependencia

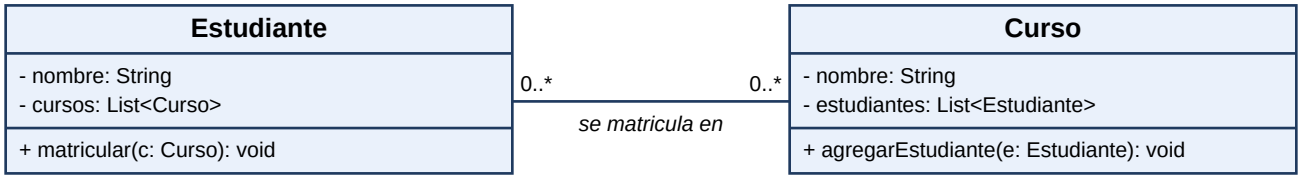
Hay dependencia cuando una clase **usa** a otra de forma temporal (como parámetro, variable local o valor de retorno) sin guardarla como atributo. Aquí **Informe** depende de **Impresora**. Se representa con flecha punteada.




```
public class Impresora {
    public void imprimir(String texto) {
        System.out.println("Imprimiendo: " + texto);
    }
}
public class Informe {
    private String contenido;

    public Informe(String contenido) { this.contenido = contenido; }

    // Dependencia: Impresora llega por parametro, no se guarda como atributo
    public void exportar(Impresora impresora) {
        impresora.imprimir(contenido);
    }
}
// Uso
new Informe("Ventas de octubre").exportar(new Impresora());
```

5. Ejemplo de asociación muchos a muchos

Un **Estudiante** puede matricularse en muchos **Cursos** y un **Curso** puede tener muchos estudiantes (multiplicidad `0..*` en ambos lados). Cada clase guarda una colección de la otra.

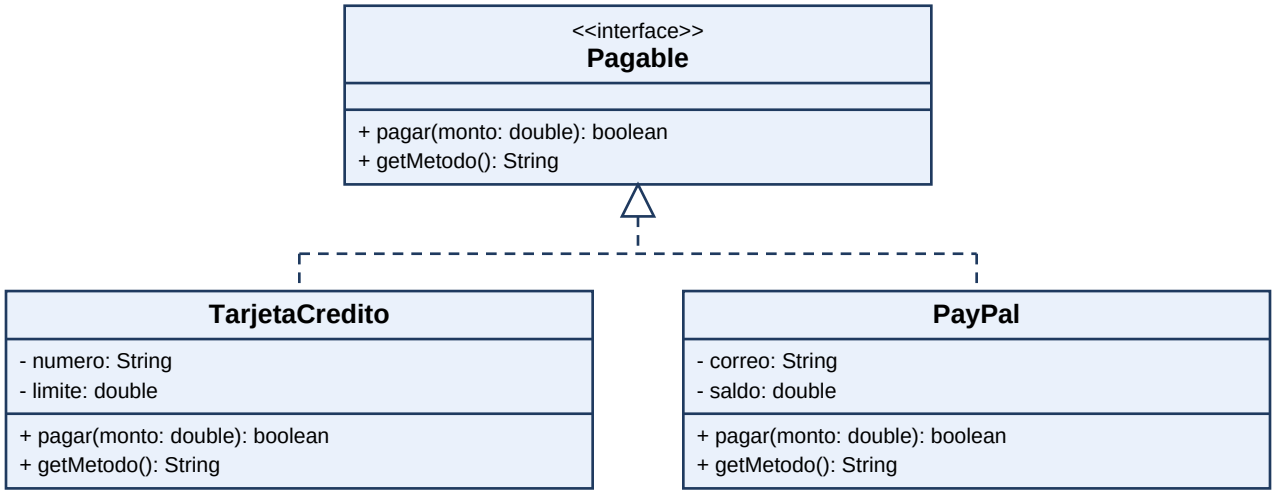



```
import java.util.ArrayList;
import java.util.List;
public class Estudiante {
    private String nombre;
    private List<Curso> cursos = new ArrayList<>();

    public Estudiante(String nombre) { this.nombre = nombre; }

    public void matricular(Curso curso) {
        if (!cursos.contains(curso)) {
            cursos.add(curso);
            curso.agregarEstudiante(this);   // mantiene ambos lados sincronizados
        }
    }
    public String getNombre() { return nombre; }
    public List<Curso> getCursos() { return cursos; }
}
```

```
public class Curso {
    private String nombre;
    private List<Estudiante> estudiantes = new ArrayList<>();

    public Curso(String nombre) { this.nombre = nombre; }

    public void agregarEstudiante(Estudiante e) {
        if (!estudiantes.contains(e)) {
            estudiantes.add(e);
            e.matricular(this);
        }
    }
    public String getNombre() { return nombre; }
    public List<Estudiante> getEstudiantes() { return estudiantes; }
}
// Uso
Estudiante ana = new Estudiante("Ana");
Estudiante luis = new Estudiante("Luis");
Curso poo = new Curso("POO");
Curso bd = new Curso("Bases de datos");

ana.matricular(poo);
ana.matricular(bd);
luis.matricular(poo);
// Ana -> 2 cursos | POO -> 2 estudiantes (Ana y Luis)
```

6. Ejemplo de interface

La interfaz **Pagable** define un contrato. **TarjetaCredito** y **PayPal** lo cumplen con `implements`, cada una a su manera. El código cliente trabaja con **Pagable** sin saber cuál es la clase concreta. En UML la realización se dibuja con línea punteada y triángulo hueco.




```
public interface Pagable {
    boolean pagar(double monto);
    String getMetodo();
}
```

```
public class TarjetaCredito implements Pagable {
    private String numero;
    private double limite;

    public TarjetaCredito(String numero, double limite) {
        this.numero = numero;
        this.limite = limite;
    }

    @Override
    public boolean pagar(double monto) {
        if (monto <= limite) { limite -= monto; return true; }
        return false;
    }

    @Override
    public String getMetodo() { return "Tarjeta de credito"; }
}
public class PayPal implements Pagable {
    private String correo;
    private double saldo;

    public PayPal(String correo, double saldo) {
        this.correo = correo;
        this.saldo = saldo;
    }

    @Override
    public boolean pagar(double monto) {
        if (monto <= saldo) { saldo -= monto; return true; }
        return false;
    }

    @Override
    public String getMetodo() { return "PayPal"; }
}
public class Main {
    public static void main(String[] args) {
        Pagable[] metodos = {
            new TarjetaCredito("1234-5678", 500000),
            new PayPal("ana@correo.com", 200000)
        };
        for (Pagable p : metodos) {
            System.out.println(p.getMetodo() + ": " + p.pagar(300000));
        }
        // Tarjeta de credito: true
        // PayPal: false
    }
}
```